

6. 정리

지금까지 필터와 인터셉터를 각각 살펴봤다. 마지막으로 두 계층의 구조적 차이를 정리하고, 언제 무엇을 선택해야 하는지, 그리고 왜 보안은 필터 계층에서 처리하는지 Spring Security의 구조를 통해 정리한다.

6.1 구조적 차이

클라이언트가 요청을 보내면 컨테이너가 요청 URI 경로에 따라 `FilterChain` 을 생성하고, 여기에는 해당 요청을 처리할 `Filter` 인스턴스들과 서블릿이 담긴다.

그리고 **Spring MVC** 애플리케이션에서 그 서블릿이 바로 `DispatcherServlet` 이다. 하나의 요청은 최대 하나의 서블릿만 처리할 수 있지만, 필터는 여러 개가 관여할 수 있다.

문서에서는 필터가 할 수 있는 일을 2가지로 요약한다.

- 하위 필터나 서블릿이 아예 호출되지 않도록 막는다. 이 경우 필터가 직접 응답을 쓴다.
- 하위 필터와 서블릿이 사용할 요청 · 응답 객체를 수정한다. : `FormContentFilter` 나 `ForwardedHeaderFilter` 가 요청을 래핑하는 것이 이 경우다.

```
public void doFilter(ServletRequest request,
    ServletResponse response, FilterChain chain) throws
    IOException, ServletException{
```

```
//나머지 애플리케이션 처리 전에 할 일
```

```

        chain.doFilter(request, response); //나머지 애플리케이션
호출
        //처리 후에 할 일
    }

```

여기서 인터셉터와의 차이가 드러난다.

	필터	인터셉터
전후 처리 방식	<code>chain.doFilter()</code> 호출을 감싸는 하나의 메서드	<code>preHandle</code> , <code>postHandle</code> , <code>afterCompletion</code> 이라는 분리된 콜백
요청 · 응답 객체 교체	<code>chain.doFilter()</code> 에 래핑한 객체를 넘겨 교체 가능	전달받은 객체를 교체할 수 없음
흐름 차단	<code>chain.doFilter()</code> 를 호출하지 않으면 된다	<code>preHandle()</code> 에서 <code>false</code> 를 반환한다

필터는 오직 하위(downstream) 필터와 서블릿에만 영향을 주기 때문에, 각 필터가 호출되는 순서가 매우 중요하다.

6.2 선택 기준

6.2.1 Filter

- `DispatcherServlet` 바깥에서 처리해야 할 때
- Spring MVC 핸들러 매핑과 무관하게 모든 요청에 적용해야 할 때
- 요청 · 응답 객체 자체를 래핑해서 수정해야 할 때
- 인코딩 · CORS · 헤더 조작처럼 HTTP 레벨의 저수준 처리가 필요한 경우
- 인증 · 인가 같은 보안 처리 (→ 6.3)

6.2.2 Interceptor

- Spring MVC 핸들러 실행 맥락이 필요할 때

- 어떤 핸들러가 요청을 처리하는지 알아야 할 때 (예: 핸들러 메서드의 어노테이션 확인)
- 핸들러 실행 전 · 후, 요청 처리 완료 후라는 세밀한 시점 제어가 필요할 때
- `WebMvcConfigurer` 로 경로 패턴을 편하게 관리하고 싶은 경우

6.3 왜 보안은 필터인가?

인터셉터는 보안 계층으로 적합하지 않으며, Spring Security 나 서블릿 필터 체인에 통합된 방식을 가능한 한 앞단에 적용하는 것이 권장된다.

Spring Security 의 서블릿 지원은 전적으로 서블릿 필터에 기반한다.

핵심은 `FilterChainProxy` 라는 특수한 필터로, `SecurityFilterChain` 을 통해 여러 보안 필터에 처리를 위임한다. (`FilterChainProxy` 는 스프링 빈이므로, 서블릿 컨테이너에 는 `DelegatingFilterProxy` 가 등록되어 요청을 이 빈에 넘겨주는 구조다.)

```
Tomcat FilterChain
├─ DelegatingFilterProxy
│   └─ FilterChainProxy (빈)
│       └─ SecurityFilterChain
│           └─ 보안 필터 0..n
```

그리고 보안 필터들은 정해진 순서로 실행되어, 각 필터가 올바른 시점에 호출됨을 보장한다.

CSRF 보호 (CsrfFilter) -> 인증(BasicAuthenticationFilter, UsernamePasswordAuthenticationFilter)
-> 인가 (AuthorizationFilter)

인증이 인가보다 먼저 실행되어야 한다는 당연한 요구를 순서로 강제하는 구조이다.

또한 `FilterChainProxy` 는 선택 사항이 아닌 작업들을 수행한다.

- 메모리 누수를 막기 위해 요청이 끝나면 `SecurityContext` 를 정리한다.
- 특정 유형의 공격으로부터 애플리케이션을 보호하기 위해 `HttpFirewall` 을 적용한다.

그리고 서블릿 컨테이너에서 필터는 URL 만으로 호출 여부가 결정되지 만, `FilterChainProxy` 는 `RequestMatcher` 인터페이스를 통해 `HttpServletRequest` 의 어떤 정보로든 어떤 `SecurityFilterChain` 을 적용할지 판단할 수 있다.

이것이 인터셉터가 대체할 수 없는 이유들이다. 인터셉터는 MVC 핸들러 매핑에 종속되어 경로 매칭 불일치 위험이 있는 반면, 보안 필터 체인은 요청이 들어오는 애플리케이션 가장 앞단에서, 일관된 매칭 규칙으로 동작한다. 또한 인터셉터는 `DispatcherServlet` 으로 가지 않는 요청(정적 리소스를 다른 서블릿이 처리하는 경우 등)에는 아예 관여하지 못한다.

6.4 커스텀 보안 필터를 만들 때

Spring Security 문서가 제시하는 **배치 규칙**이 유용하다. 필터 체인의 주요 이벤트는 다음 순서로 일어난다.

SecurityContext 로드 -> 공격으로 요청 보호 -> 인증 -> 인가

1. 세션 등에서 SecurityContext 를 로드한다.
2. 보안 헤더, CORS, CSRF 같은 일반적인 공격으로부터 요청을 보호한다.
3. 요청을 인증한다.
4. 요청을 인가한다.

그래서 커스텀 필터는 성격에 따라 다음 위치에 둔다.

필터 성격	배치 위치
취약점 보호 필터	SecurityContextHolderFilter 뒤
인증 필터	LogoutFilter 뒤
인가 필터	AnonymousAuthenticationFilter 뒤

```
@Bean
SecurityFilterChain filterChain(HttpSecurity http)
throws Exception {
    http
        // ...
        .addFilterAfter(new TenantFilter(),
            AnonymousAuthenticationFilter.class);
    return http.build();
}
```

필터를 빈으로 선언할 때 주의점

필터를 `@Component` 나 빈으로 선언하면 **Spring Boot** 가 임베디드 컨테이너에 자동 등록한다. 그러면 컨테이너가 한 번, **Spring Security** 가 한 번, 총 두 번 호출될 수 있고, 순서도 서로 달라진다. 그래서 보안 필터는 종종 빈으로 만들지 않는다.

다만 의존성 주입 등을 위해 빈이어야 한다

면, `FilterRegistrationBean` 을 선언하고 `enabled` 속성을 `false` 로 설정해 컨테이너 등록을 막으면 된다.

```
@Bean
public FilterRegistrationBean<TenantFilter>
tenantFilterRegistration(TenantFilter filter) {
    FilterRegistrationBean<TenantFilter> registration =
    new FilterRegistrationBean<>(filter);
    registration.setEnabled(false);
    return registration;
}
```

Spring Security 문서 역시 커스텀 필터 작성 시 `Filter` 를 직접 구현하는 대신, 요청당 한 번 호출되는 필터의 베이스 클래스

인 `OncePerRequestFilter` 를 상속하기를 권한

다. `OncePerRequestFilter` 는 `HttpServletRequest`, `HttpServletResponse` 파라미터를 받는 `doFilterInternal()` 메서드를 제공하므로, `ServletRequest` 를 캐스팅할 필요도 없다.

후기

1~6 챕터로 나누어 Spring 공식 문서(Spring Framework, Spring Security)를 Claude 와 함께 조금씩 훑어보았다. 처음 Spring Boot 를 공부했을 땐 "이렇게 쓰면 되는 건가?", "이렇게 하면 편하구나"에 그쳤지만, 현재는 내부 동작 방식에 대해서 조금씩 공부해보고 있는데 확실히 내

기준으로는 난이도가 어려웠다. 물론 나의 기본기가 부족하다는 것도 인지하고 있다. 한 번에 이해되는 게 없어서 따로 검색해보는 일이 상당히 잦았다.

사실 이건 몇 달 전에 훑어본 내용이고, 최종적인 문서화는 오늘이 되어야 진행하게 되었다. (~~아런저런 일에 치이며 마루다 보나..~~) 지금 글을 쓰면서 느낀 건데, 다시 꼭 복습을 해야겠다는 생각이 엄청나게 든다..

AI 덕분에 예전에는 만들지 못했던 것들도 만들고, 코드 생산량도 크게 늘면서 일상에 편함을 가져다주었다. 하지만 오히려 그 편함 때문에 예전만큼의 의지가 보이지 않는 것 같기도 하다. 그래서 이렇게 공부하는 것이 어떻게 보면 살아남기 위한 작은 발버둥 같다는 생각도 든다.